

RELATÓRIO – SIMULADOR DE ESCALONAMENTO DE PROCESSOS

1. Introdução

Este trabalho teve como objetivo desenvolver um simulador de escalonamento de processos inspirado nos mecanismos utilizados por Sistemas Operacionais modernos. O projeto busca demonstrar, de forma prática, conceitos fundamentais da disciplina, como gerenciamento de processos, escalonamento de CPU, preempção, bloqueio por operações de Entrada e Saída (I/O) e controle de prioridades.

Em um ambiente multiprogramado, diversos processos competem simultaneamente pelo uso do processador. Cabe ao escalonador decidir qual processo deve utilizar a CPU em cada instante, buscando equilibrar desempenho, justiça e tempo de resposta. Para representar esse cenário, foi implementado um simulador baseado em relógio lógico (clock), permitindo acompanhar a evolução do sistema ao longo do tempo e observar como diferentes eventos afetam a execução dos processos.

2. Descrição do Algoritmo

O algoritmo escolhido foi o Round Robin com Feedback, uma estratégia amplamente utilizada por combinar simplicidade, justiça e adaptação ao comportamento dos processos.

A implementação utiliza duas filas de processos prontos:

- Fila de Alta Prioridade;
- Fila de Baixa Prioridade;

O escalonador sempre prioriza a execução dos processos presentes na fila de alta prioridade. Somente quando essa fila estiver vazia os processos da fila de baixa prioridade são atendidos.

O mecanismo de feedback foi implementado da seguinte forma:

Rebaixamento de prioridade

Quando um processo utiliza integralmente o quantum de CPU sem finalizar sua execução nem solicitar I/O, entende-se que ele possui comportamento predominantemente orientado à CPU (CPU-bound). Nesse caso, o processo sofre preempção e retorna para a fila de baixa prioridade.

Retorno após I/O

Quando um processo solicita uma operação de entrada e saída antes de consumir todo o quantum, ele libera voluntariamente a CPU. Após concluir a operação de I/O, sua prioridade é redefinida conforme o dispositivo utilizado.

Essa política favorece processos interativos e evita que processos que frequentemente realizam I/O fiquem aguardando excessivamente pela CPU.

3. Premissas

Para manter o comportamento da simulação controlado e reproduzível, foram adotadas as seguintes premissas:

Quantum

O quantum foi definido como:

- 3 unidades de tempo

Esse valor permite observar facilmente os efeitos da preempção e da realimentação de prioridades.

Processos

São gerados automaticamente:

- 5 processos iniciais

Cada processo recebe:

- PID único;
- Prioridade inicial alta;
- Tempo total de CPU aleatório;
- Possível evento de I/O.

Tempo de CPU

O tempo necessário para conclusão de cada processo é sorteado entre:

- Mínimo: 4 unidades;
- Máximo: 12 unidades

Operações de I/O

Os dispositivos simulados são:

Dispositivo	Tempo de Bloqueio
Disco	4
Fita	5
Impressora	8
Nenhum	0

Cada processo pode realizar no máximo uma operação de I/O durante sua execução.

Política de retorno após I/O

Foi adotada a seguinte estratégia:

Dispositivo	Prioridade ao Retornar
Disco	Baixa
Fita	Alta
Impressora	Alta
Nenhum	Alta

Essa decisão busca demonstrar que diferentes tipos de dispositivos podem influenciar de forma distinta o comportamento do escalonador.

Semente Aleatória

Foi utilizada a semente:

SEED = 42

Isso garante que a geração dos processos seja determinística, permitindo reproduzir exatamente os mesmos resultados em diferentes execuções.

4. Estruturas de Dados

PCB (Process Control Block)

Cada processo foi representado por um dicionário Python, funcionando como uma versão simplificada de um PCB real.

Entre as informações armazenadas estão:

- PID e PPID;
- Estado do processo;
- Prioridade atual;
- Tempo restante de CPU;
- Tempo total de CPU;
- Dados relacionados ao I/O;
- Estatísticas de execução;
- Quantidade de preempções sofridas.

Filas

As filas foram implementadas utilizando listas nativas da linguagem Python.

Foram utilizadas três estruturas principais:

- fila_alta;
- fila_baixa;
- fila_io;

As filas de prioridade seguem o comportamento FIFO (First In, First Out), preservando a ordem de chegada dos processos.

A fila de I/O armazena os processos bloqueados enquanto aguardam a conclusão de suas operações de entrada e saída.

Lista de Finalizados

Os processos concluídos são armazenados em uma lista separada chamada finalizados, utilizada posteriormente para cálculo das métricas finais.

5. Fluxo de Execução

A simulação é baseada em um relógio lógico. Cada iteração do laço principal representa uma unidade de tempo do sistema.

Inicialização

Os processos são criados automaticamente e inseridos na fila correspondente à sua prioridade inicial.

Todos começam no estado:

- NOVO

e são imediatamente movidos para:

- PRONTO

Escalonamento

Sempre que a CPU fica livre, o escalonador procura processos na fila de alta prioridade.

Caso não existam processos nessa fila, a CPU passa a atender a fila de baixa prioridade.

Execução

Enquanto ocupa a CPU, o processo tem:

- Seu tempo restante reduzido;
- Seu quantum reduzido;

Bloqueio por I/O

Quando o processo atinge o ponto programado para solicitar I/O:

- Deixa a CPU;
- Muda para o estado BLOQUEADO;
- Recebe o tempo correspondente ao dispositivo;
- Entra na fila de I/O;

Enquanto isso, outros processos podem continuar utilizando a CPU normalmente.

Conclusão do I/O

Os processos bloqueados têm seus tempos de espera reduzidos paralelamente ao processamento da CPU.

Ao concluir a operação:

- Retornam ao estado PRONTO;
- Recebem a prioridade definida pela política de feedback;
- São inseridos na fila apropriada;

Preempção

Quando o quantum chega a zero:

- O processo é removido da CPU;
- Sofre uma preempção;
- É rebaixado para a fila de baixa prioridade;

Finalização

Quando o tempo restante de CPU chega a zero:

- O processo muda para o estado FINALIZADO;
- Registra seu instante de término;
- É armazenado na lista de histórico;

A simulação termina quando não existem mais processos em execução, prontos ou bloqueados.

6. Execução

O projeto foi desenvolvido em Python 3 e utiliza exclusivamente bibliotecas padrão da linguagem.

Requisitos

- Python 3.8 ou superior

Execução local

No terminal, acessar a pasta do projeto e executar:

```
python3 src/main.py
```

Dependendo do sistema operacional, também pode ser utilizado:

```
python src/main.py
```

Execução com Docker

Além da execução local com Python, o projeto também foi containerizado com Docker. Foi criado um Dockerfile funcional, permitindo construir uma imagem do

simulador e executá-lo em um ambiente padronizado, sem necessidade de configurar manualmente a versão do Python na máquina.

Para construir a imagem Docker:

```
docker build -t so-escalador-grupo-2 .
```

Para executar o simulador dentro do container:

```
docker run --rm so-escalador-grupo-2
```

O projeto utiliza um arquivo .gitignore para evitar o versionamento de:

- Ambientes virtuais;
- Arquivos temporários;
- Caches do Python;
- Arquivos gerados automaticamente pelo sistema operacional;

Criação inicial dos processos gerados pelo simulador:

```
jvkrabbe@Krabbe:~/projetos/sisop-round-robin$ python3 src/main.py  
[config] quantum=3 | processos=5 | seed=42
```

```
--- INICIANDO SIMULAÇÃO ROUND ROBIN COM FEEDBACK ---
```

```
[t=000] [CRIACAO] Processo 1 criado -> fila ALTA | cpu=6 | io=FITA@1  
[t=000] [CRIACAO] Processo 2 criado -> fila ALTA | cpu=12 | io=NENHUM@-1  
[t=000] [CRIACAO] Processo 3 criado -> fila ALTA | cpu=5 | io=DISCO@4  
[t=000] [CRIACAO] Processo 4 criado -> fila ALTA | cpu=4 | io=IMPRESSORA@1  
[t=000] [CRIACAO] Processo 5 criado -> fila ALTA | cpu=5 | io=DISCO@2
```

7. Saída do Simulador

Durante a execução, o programa produz logs detalhados de cada evento relevante.

Registro dos eventos de execução, preempção e operações de I/O

```
[t=000] [ESCALONADOR] Processo 1 (ALTA) entrou na CPU.
[t=003] [CPU] Processo 1 esgotou Quantum. [FEEDBACK: BAIXA Prio]
[t=003] [ESCALONADOR] Processo 2 (ALTA) entrou na CPU.
[t=006] [CPU] Processo 2 esgotou Quantum. [FEEDBACK: BAIXA Prio]
[t=006] [ESCALONADOR] Processo 3 (ALTA) entrou na CPU.
[t=007] [CPU] Processo 3 pediu I/O (DISCO - 4u). Foi para BLOQUEADO.
[t=007] [ESCALONADOR] Processo 4 (ALTA) entrou na CPU.
[t=010] [CPU] Processo 4 pediu I/O (IMPRESSORA - 8u). Foi para BLOQUEADO.
[t=010] [ESCALONADOR] Processo 5 (ALTA) entrou na CPU.
[t=011] [I/O] Processo 3 terminou uso de DISCO. [FEEDBACK: BAIXA Prio]
[t=013] [CPU] Processo 5 pediu I/O (DISCO - 4u). Foi para BLOQUEADO.
[t=013] [ESCALONADOR] Processo 1 (BAIXA) entrou na CPU.
[t=015] [CPU] Processo 1 pediu I/O (FITA - 5u). Foi para BLOQUEADO.
[t=015] [ESCALONADOR] Processo 2 (BAIXA) entrou na CPU.
[t=017] [I/O] Processo 5 terminou uso de DISCO. [FEEDBACK: BAIXA Prio]
[t=018] [I/O] Processo 4 terminou uso de IMPRESSORA. [FEEDBACK: ALTA Prio]
[t=018] [CPU] Processo 2 esgotou Quantum. [FEEDBACK: BAIXA Prio]
[t=018] [ESCALONADOR] Processo 4 (ALTA) entrou na CPU.
[t=019] [CPU] Processo 4 FINALIZADO.
[t=019] [ESCALONADOR] Processo 3 (BAIXA) entrou na CPU.
[t=020] [I/O] Processo 1 terminou uso de FITA. [FEEDBACK: ALTA Prio]
[t=022] [CPU] Processo 3 esgotou Quantum. [FEEDBACK: BAIXA Prio]
[t=022] [ESCALONADOR] Processo 1 (ALTA) entrou na CPU.
[t=023] [CPU] Processo 1 FINALIZADO.
[t=023] [ESCALONADOR] Processo 5 (BAIXA) entrou na CPU.
[t=025] [CPU] Processo 5 FINALIZADO.
[t=025] [ESCALONADOR] Processo 2 (BAIXA) entrou na CPU.
[t=028] [CPU] Processo 2 esgotou Quantum. [FEEDBACK: BAIXA Prio]
[t=028] [ESCALONADOR] Processo 3 (BAIXA) entrou na CPU.
[t=029] [CPU] Processo 3 FINALIZADO.
[t=029] [ESCALONADOR] Processo 2 (BAIXA) entrou na CPU.
[t=032] [CPU] Processo 2 FINALIZADO.
```

Interpretação da execução

A imagem acima apresenta o comportamento completo do escalonador durante a execução do simulador. É possível observar a alternância entre processos, a ocorrência de preempções por esgotamento do quantum e os eventos de entrada e saída (I/O), demonstrando o funcionamento do mecanismo de feedback implementado.

Ao final da simulação são exibidas métricas importantes.
Estatísticas finais da execução do simulador:


```

--- SIMULAÇÃO CONCLUÍDA EM 32 UNIDADES DE TEMPO ---
Processos finalizados: 5/5
Preempções: 5
Eventos de I/O: DISCO=2 | FITA=1 | IMPRESSORA=1
CPU ociosa: 0u (0.00%)
Tempo médio de espera: 15.00u
Turnaround médio: 25.60u

```

PID	PPID	PRIO FINAL	STATUS	ESPERA	TURNAROUND
1	0	ALTA	FINALIZADO	12	23
2	0	BAIXA	FINALIZADO	20	32
3	0	BAIXA	FINALIZADO	20	29
4	0	ALTA	FINALIZADO	7	19
5	0	BAIXA	FINALIZADO	16	25

- Quantidade de processos finalizados;
- Número total de preempções;
- Quantidade de eventos de I/O;
- Percentual de CPU ociosa;
- Tempo médio de espera;
- Turnaround médio;
- Estatísticas individuais por processo;

Essas métricas permitem avaliar o comportamento do algoritmo e sua eficiência.

Observa-se que todos os cinco processos foram concluídos com sucesso, sem períodos de ociosidade da CPU, indicando aproveitamento integral do processador.

7.1 Análise dos resultados

Foram registradas cinco preempções, resultado esperado devido ao quantum de três unidades de tempo adotado pelo algoritmo Round Robin. Os processos que realizaram operações de I/O retornaram às filas de acordo com a política de feedback implementada, demonstrando o funcionamento correto do mecanismo de priorização. O tempo médio de espera obtido foi de 15 unidades de tempo, enquanto o turnaround médio foi de 25,6 unidades, valores compatíveis com a carga de trabalho simulada.

8. Conclusão

O desenvolvimento do simulador permitiu aplicar na prática diversos conceitos estudados em Sistemas Operacionais, especialmente aqueles relacionados ao gerenciamento de processos e escalonamento de CPU.

A principal contribuição do projeto foi demonstrar como diferentes comportamentos dos processos influenciam diretamente as decisões do escalonador. O mecanismo de feedback mostrou-se eficiente para diferenciar processos mais dependentes de CPU daqueles que realizam operações frequentes de entrada e saída.

Uma das maiores dificuldades encontradas foi representar a execução paralela das operações de I/O sem utilizar múltiplas threads. Essa limitação foi superada por meio de um relógio lógico centralizado, responsável por atualizar simultaneamente o estado da CPU e dos dispositivos de I/O a cada ciclo.

Como possíveis melhorias futuras, destacam-se:

- Inclusão de múltiplos processadores;
- Implementação de mais níveis de prioridade;
- Simulação de gerenciamento de memória;
- Geração automática de gráficos de desempenho;
- Exportação de relatórios em formato CSV ou PDF;
- Comparação entre diferentes algoritmos de escalonamento;

Essas extensões tornariam a simulação ainda mais próxima do funcionamento de um Sistema Operacional real.